



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

Práctica 1

Materia: Matemáticas por computadora
Alumno: Fabian Ruiz Luna
Catedrático: Luis Jesus Padrón Padrón
Carrera: Ingeniería Mecatronica
Semestre: 4to

Rioverde, S.L.P.
16 de Enero del 2026

Índice

1. Introducción	1
2. Parte teorica	2
3. Ejercicios practicos	4

1. Introducción

En el presente reporte, se presentaran distintos ejercicios de libro "Numerical Methods" evidencias de las respuestas.

Los ejercicios van desde el número 2.51 hasta el 2.65, donde, en caso de tener que presentar un código Python, se presentará para su evidencia.

2. Parte teorica

Ejercicio 2.51

"Let $f(x)$ be a continuous function on the interval $[a,b]$ where $f(a) \cdot f(b) < 0$. Clearly give all of the mathematical details for how the Bisection Method approximates the root of the function $f(x)$ in the interval $[a,b]$ ". En el siguiente ejercicio se solicita, simplemente explicar y describir el fundamento matemático del método de bisección

2.51 Let $f(x)$ be a continuous function on the interval $[a,b]$ where $f(a) \cdot f(b) < 0$. Clearly give all of the mathematical details for how the Bisection Method approximates the root of the function $f(x)$ in the interval $[a,b]$.

$$\log_2\left(\frac{B-A}{\epsilon}\right) - 1 = n \rightarrow \text{numero de iteraciones}$$

$$\frac{B-A}{2^{n+1}} [B-A] \rightarrow \text{Intervalos}$$

Figura 1: Respuesta

Ahora una explicación más detallada y puesta en palabras de mi respuesta previamente mostrada es que, guiándonos en base al Teorema del valor intermedio, el cual dicta que una función continua en un intervalo entre $[a,b]$, entonces, existe un valor C que represente el valor medio en el intervalo, y aquí se aplica a través la fórmula mostrada, que, resta los valores y los divide a la mitad, permitiendo acercarnos a el valor C , puesto de manera muy resumida y un tanto burda.

Ejercicio 2.52

"Let $f(x)$ be a continuous function on the interval $[a,b]$ where $f(a) \cdot f(b) < 0$. Clearly give all of the mathematical details for how the Regula Falsi Method approximates the root of the function $f(x)$ in the interval $[a,b]$ ". En el siguiente ejercicio se pide explicar detalles matemáticos de cómo el método de Falsa Posición se aproxima a la raíz entre $[a,b]$

2.52 $f(a) \cdot f(b) < 0, [a,b]$, falsa posición

$$x_r = b - \frac{f(b)(b-a)}{f(b) - f(a)}$$

Se aproxima a traves de

- Calcular x_r
- Evaluar $f(x_r)$
- Revisar el signo
- Si $f(a) \cdot f(x_r) < 0$
- Si $f(b) \cdot f(x_r) < 0$
- Repetir el proceso

Figura 2: Respuesta

Ahora, explicando esto como es debido la falsa posición traza una línea recta que une los puntos $(a, f(a))$ y $(b, f(b))$. El lugar donde esa recta cruza el eje X se convierte en nuestra nueva aproximación x_r .

Ejercicio 2.53 "Let $f(x)$ be a differentiable function with a root near $x = x_0$. Clearly give all of the mathematical details for how Newton's Method approximates the root of the function $f(x)$ ". Ahora en este ejercicios se pide explicar de donde sale la formula y como se usa iterativamente.

Handwritten derivation of Newton's Method formula on graph paper:

$$\begin{aligned}
 2.53: f(x) \\
 0 &= f(x_0) + f'(x_0)(x - x_0) \\
 -f(x_0) &= f'(x_0)(x - x_0) \\
 x - x_0 &= \frac{-f(x_0)}{f'(x_0)} \\
 x &= x_0 - \frac{f(x_0)}{f'(x_0)} \\
 x_1 &= x_0 - \frac{f(x_0)}{f'(x_0)}
 \end{aligned}$$

Figura 3: Respuesta

Ahora, explicando este proceso en mayor medida en el metodo de Newton Raphson traza una recta tangente a la curva en ese punto y ve dónde esa recta cruza el eje x . Ese punto de cruce es tu siguiente mejor estimación (x_1).

Ejercicio 2.54 "Let $f(x)$ be a continuous function with a root near $x = x_0$. Clearly give all of the mathematical details for how the Secant Method approximates the root of the function $f(x)$ ". En este ultimo ejercicio se pide explicar esta versión modificada de Newton Rapshon, en la imagen proporcionada se toma desde el lado practico.

Handwritten derivation of the Secant Method formula on graph paper:

$$\begin{aligned}
 2.54 \\
 m &= \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}} \\
 y - f(x_n) &= m(x - x_n) \\
 y &= 0 \\
 0 - f(x_n) &= m(x_{n+1} - x_n) \\
 -f(x_n) &= m(x_{n+1} - x_n) \\
 x_{n+1} - x_n &= \frac{-f(x_n)}{m} \\
 x_{n+1} &= x_n - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}
 \end{aligned}$$

Figura 4: Respuesta

Ahora explicando esta formula fuera, de la practica, esta nace a partir de el hecho de que Newton Raphson puede fallar cuando la raíz tiene multiplicidad, graficamente seria una recta tangente casi horizaontal

3. Ejercicios practicos

Ahora, procediendo a la segunda parte de ejercicios, siendo estos puramente practicos.

Ejercicio 2.55 "How many iterations of the bisection method are necessary to approximate $\sqrt{3}$ to within $10^{-3}, 10^{-4}, \dots, 10^{-15}$ using the initial interval $[a, b] = [0, 2]$? See Theorem 2.2.. En este se pide el numero de iteraciones para llegar hasta la raíz, y haciendo uso de codigo en Python.

```

1 # Función bisección
2 import math
3
4 def biseccion(f, a, b, tol):
5     """Calcula la raíz de la función f en el intervalo [a, b] con una tolerancia de tol.
6     """
7     # Verificar que la función sea continua en el intervalo
8     if f(a) * f(b) >= 0:
9         raise ValueError("La función no cambia de signo en el intervalo [a, b].")
10
11     # Inicialización
12     c = (a + b) / 2
13     iteraciones = 0
14
15     while abs(f(c)) > tol:
16         # Actualización del intervalo
17         if f(a) * f(c) < 0:
18             b = c
19         else:
20             a = c
21         c = (a + b) / 2
22         iteraciones += 1
23
24     return c, iteraciones
25
26 # Ejemplo de uso
27 f = lambda x: x**2 - 3
28 a, b = 0, 2
29 tol = 1e-15
30
31 c, iteraciones = biseccion(f, a, b, tol)
32
33 print(f"Raíz aproximada: {c}")
34 print(f"Número de iteraciones: {iteraciones}")

```

Figura 5: Código

El numero total de iteraciones para este ejercicio es de 49 iteraciones

Ejercicio 2.56

Refer back to Example 2.1 and demonstrate that you get the same results by solving the problem $x^3 - 3 = 0$. Generate versions of all of the plots from the Example and give thorough descriptions of what you learn from each plot". En este ejercicio se pide, analizar el comportamiento de la función previamente mencionada

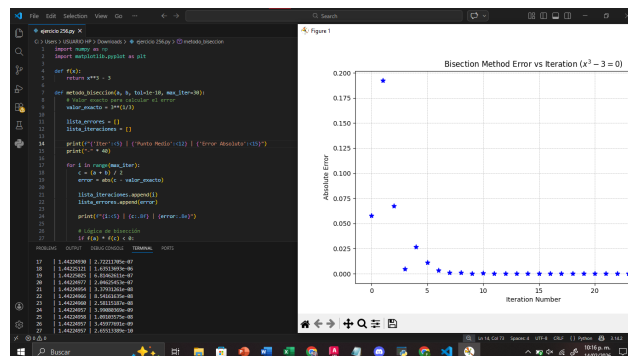


Figura 6: Código

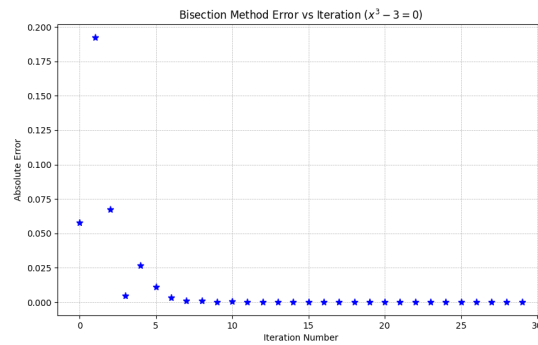


Figura 7: Código

Ejercicio 2.57

In this problem you will demonstrate that all of your root finding codes work. At the beginning of this chapter we proposed the equation solving problem

$$3 \sin(x) + 9 = x^2 - \cos(x).$$

Write a script that calls upon your Bisection, Regula Falsi, Newton, and Secant methods one at a time to find the positive solution to this equation. Your script needs to output the solutions in a clear and readable way so you can tell which answer can from which root finding algorithm..^{En este ejercici se pide resolver la siguiente función a través de varios metodos}

```

% Ejercicio 2.57: Resolver la ecuación 3 sin(x) + 9 = x^2 - cos(x)
% Usando el método de Bisección

% Definición de la función f(x)
f = @(x) 3*sin(x) + 9 - (x^2 - cos(x));

% Intervalo de búsqueda [a, b]
a = 0;
b = 10;

% Verificación de que f(a) y f(b) tienen signos opuestos
if f(a)*f(b) > 0
    error('La función no cambia de signo en el intervalo [a, b].');
end

% Aplicación del método de Bisección
for i = 1:100
    c = (a+b)/2;
    if f(c) == 0
        break;
    end
    if f(a)*f(c) < 0
        b = c;
    else
        a = c;
    end
end

% Solución encontrada
sol = c;

% Salida
fprintf('La solución encontrada es: %f\n', sol);

```

Figura 8: Código

```

def metodo_secante(f, x0):
    """Metodo de la secante para encontrar raices de una funcion f(x)."""
    # Verificar que la funcion f sea callable
    if not callable(f):
        raise ValueError("La funcion f no es callable")
    # Verificar que x0 sea un numero
    if not isinstance(x0, (int, float)):
        raise ValueError("El valor inicial x0 no es un numero")
    # Inicializar variables
    x1 = x0 + 1
    error = 1
    iteraciones = 0
    # Bucle principal
    while error > 1e-6:
        # Calcular f(x0) y f(x1)
        f0 = f(x0)
        f1 = f(x1)
        # Verificar que f(x0) y f(x1) no sean cero
        if f0 == 0 or f1 == 0:
            raise ValueError("La funcion f tiene raices en x0 o x1")
        # Calcular el siguiente punto x2
        x2 = x1 - f1 * (x1 - x0) / (f1 - f0)
        # Calcular el error
        error = abs(f1)
        # Actualizar x0 y x1
        x0 = x1
        x1 = x2
        iteraciones += 1
    # Retornar el resultado
    return x1, iteraciones

```

Figura 9:Codigo

Eercicio 2.58

a) If we consider the equation $|x_{k+1} - x_*| \leq C|x_k - x_*|^M$ and take the logarithm (base 10) of both sides then we get:

$$\log(|x_{k+1} - x_*|) \leq \log(C) + M \log(|x_k - x_*|)$$

”

En este ejercicio se pide basicamente, mostrar la formula antes presentada, pero cuando se le aplican las propiedad de los logaritmos

b) In part (a) you should have found that the log of new error is a linear function of the log of the old error. What is the slope of this linear function on a log-log plot?”

c) In the plots below you will see six different log-log plots of the new error to the old error for different root finding techniques. What is the order of the approximate convergence rate for each of these methods?”

d) In your own words, what does it mean for a root finding method to have a “first order convergence rate?” “Second order convergence rate?” etc.”

```

def metodo_secante(f, x0):
    """Metodo de la secante para encontrar raices de una funcion f(x)."""
    # Verificar que la funcion f sea callable
    if not callable(f):
        raise ValueError("La funcion f no es callable")
    # Verificar que x0 sea un numero
    if not isinstance(x0, (int, float)):
        raise ValueError("El valor inicial x0 no es un numero")
    # Inicializar variables
    x1 = x0 + 1
    error = 1
    iteraciones = 0
    # Bucle principal
    while error > 1e-6:
        # Calcular f(x0) y f(x1)
        f0 = f(x0)
        f1 = f(x1)
        # Verificar que f(x0) y f(x1) no sean cero
        if f0 == 0 or f1 == 0:
            raise ValueError("La funcion f tiene raices en x0 o x1")
        # Calcular el siguiente punto x2
        x2 = x1 - f1 * (x1 - x0) / (f1 - f0)
        # Calcular el error
        error = abs(f1)
        # Actualizar x0 y x1
        x0 = x1
        x1 = x2
        iteraciones += 1
    # Retornar el resultado
    return x1, iteraciones

```

Figura 10:Codigo

Ejercicio 2.59

”Shelby started using Newton’s method to solve a root-finding problem. To test her code she was using an equation for which she knew the solution. Given the starting point the absolute error after one step of Newton’s method was $|x_1 - x_*| = 0.2$. What is the approximate expected error at step 2? What about at step 3? Step 4? Defend your answers by fully describing your thought process”.En este ejercicio se nos pide encontrar el error esperado y defender.

$$\begin{aligned}
 2.59 \quad |x_1 - x_0|/2 \\
 e_2 \approx (0.2)^2 = 0.04 \\
 e_3 \approx (0.04)^2 = 0.0016 \\
 e_4 \approx (0.0016)^2 = 0.0000256 \\
 \text{El método de Newton tiene un orden de convergencia } \alpha=2
 \end{aligned}$$

Figura 11: Código

Ejercicio 2.60

- Solve for x in the case that $N = 2$. Then write a Python function that implements this root-finding method.
- Demonstrate that your code from part (a) is indeed working by solving several problems where you know the exact solution.
- Show several plots that estimates the order of the method from part (a). That is, create a log-log plot of the successive errors for several different equation-solving problems.
- What are the pro's and con's to using this new method?

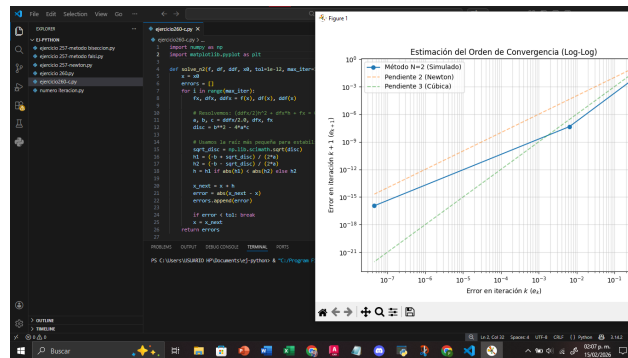


Figura 12: Código

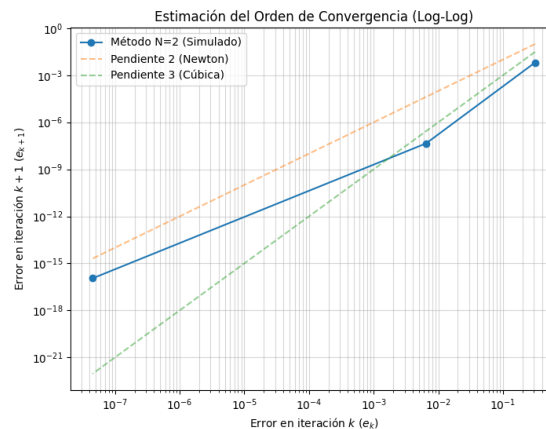


Figura 13: Código

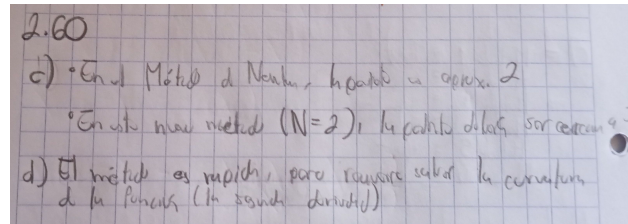


Figura 14: Código

Ejercicio 2.61

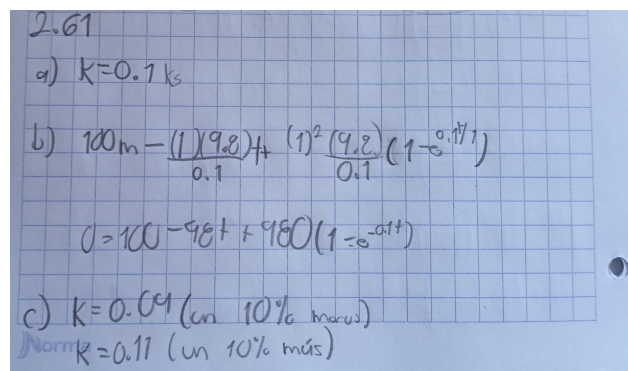
a) What are the units of the parameter k ?b) If $m = 1\text{kg}$, $g = 9.8\text{m/s}^2$, $k = 0.1$, and $s_0 = 100\text{m}$ how long will it take for the object to hit the ground? Find your answer to within 0.01 seconds.c. The value of k depends on the aerodynamics of the object and might be challenging to measure. We want to perform a sensitivity analysis on your answer to part (b) subject to small measurement errors in k . If the value of k is only known to within 10% then what are your estimates of when the object will hit the ground?

Figura 15: Código

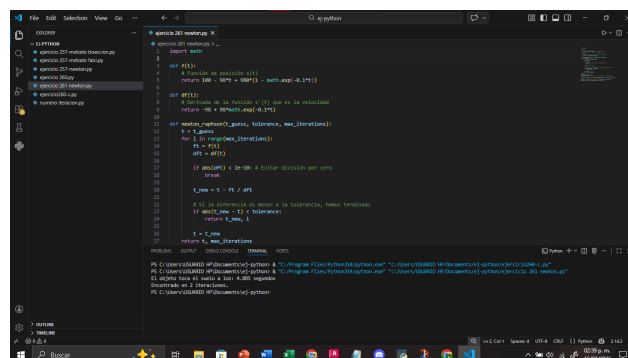


Figura 16: Código

Ejercicio 2.62

Can the Bisection Method, Regula Falsi Method, or Newton's Method be used to find the roots of the function $f(x) = \cos(x) + 1$? Explain why or why not for each technique".

R: Se puede usar Bisección?, no, porque requiere que la función cambie, sin embargo esta no tiene cambio alguna. Ahora, se puede usar Falsa posición?, tampoco?, por lo mismo del anterior, pero con un origen diferente, y es por tomar el Teorema del valor intermedio. Y por ultimo, se puede usar Newton?, si, ya que esta no depende de un cambio

Ejercicio 2.63

Exercise 2.63 In Single Variable Calculus you studied methods for finding local and global extrema of functions. You likely recall that part of the process is to set the first derivative to zero and to solve for the independent variable (remind yourself why you're doing this). The trouble with this process is that it may be very very challenging to solve by hand. This is a perfect place for Newton's method or any other root finding technique!

Find the local extrema for the function $f(x) = x^3(x - 3)(x - 6)^4$ using numerical techniques where appropriate.

```

1 # Definición de la función f(x) calculada analíticamente a por software
2 f = lambda x: x**3 * (x - 3)**1 * (x - 6)**4
3 def df(x):
4     return 3*x**2 * (x - 3)**1 * (x**2 - 12*x + 36) + 4*x**3 * (x - 6)**3
5
6 # Definición de la segunda derivada f''(x) para el método de Newton
7 def d2f(x):
8     return 6*x * (x - 3)**1 * (x**2 - 12*x + 36) + 12*x**2 * (x - 6)**2
9
10 # Método de Newton para encontrar raíces
11 def newton_method(guess, tolerance=1e-7, max_iterations=100):
12     x = guess
13     for i in range(max_iterations):
14         fx = f(x)
15         dfx = df(x)
16         if abs(fx) < tolerance: break # Evitar división por cero
17         x_new = x - fx / dfx
18     return x_new
19
20 # Ejecución de los métodos de búsqueda de raíces
21 # Punto fijo encontrado en: 1.8
22 # Punto fijo encontrado en: 1.8
23 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
24 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
25 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
26 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
27 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
28 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
29 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
30 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
31 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
32 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
33 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
34 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
35 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
36 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
37 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
38 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
39 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
40 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
41 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
42 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
43 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
44 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
45 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
46 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
47 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
48 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
49 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
50 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
51 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
52 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
53 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
54 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
55 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
56 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
57 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
58 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
59 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
60 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
61 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
62 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
63 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
64 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
65 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
66 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
67 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
68 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
69 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
70 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
71 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
72 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
73 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
74 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
75 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
76 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
77 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
78 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
79 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
80 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
81 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
82 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
83 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
84 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
85 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
86 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
87 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
88 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
89 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
90 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
91 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
92 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
93 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
94 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
95 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
96 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
97 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
98 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
99 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]
100 # Punto crítico encontrado (donde f'(x)=0): [0, 1.125, 5.9375, 6]

```

Figura 17: Código

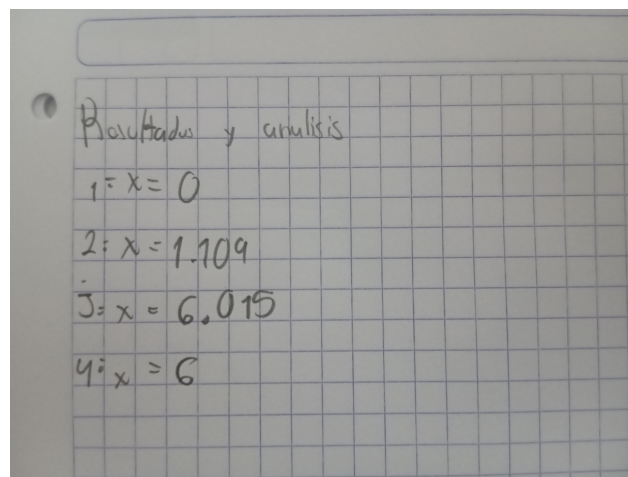


Figura 18: Foto